



What is Columna (datumwise.ai)?

Columna (by Datumwise) is a data knowledge and documentation platform that sits on top of your analytics stack and automatically generates, organizes, and maintains human-readable documentation for your tables and columns using AI.^[1]

What Columna Is

Columna is essentially a “self-documenting semantic layer” product from Datumwise/Datawise that focuses on:

- Connecting to your existing data warehouse or lake (e.g., Bronze/Silver views in a modern data stack).
- Using AI to describe tables and columns in plain language.
- Suggesting governance and classification labels (PII, Finance, Certified, etc.) based on patterns in the data.
- Propagating metadata and documentation downstream as views reference each other.^[1]

The goal is to reduce data chaos and make it easier for analysts, engineers, and business users to understand what each dataset means without maintaining separate, manual documentation projects.^[1]

Key capabilities

From Datumwise’s description of the self-documenting semantic layer that underlies Columna:^[1]

- **AI-generated descriptions:** Samples data and automatically writes human-readable descriptions for each column and table.
- **Automated label suggestions:** Analyzes names, types, and value patterns to propose governance labels (e.g., PII, financial data).
- **Metadata propagation:** When a downstream view references an upstream one (Silver → Bronze), the column descriptions and labels flow through, so documentation is written once and reused everywhere.

Why teams use it

Teams use Columna to:

- Cut down the time spent writing and maintaining data documentation.
- Improve discoverability and trust in datasets across analytics and AI projects.
- Enforce consistent governance and semantic meaning across multiple layers of their data platform.^[1]

If you share a bit about your stack (warehouse, BI tools, size of team), I can outline how Columna would fit into it and what workflows it might replace.

✱

Is datumwise a company?

Yes. Datumwise (often branded as Datawise) is an information technology and services company focused on AI, machine learning, and data engineering solutions.^[10] ^[11]

Company overview

- **Name/brand:** Operates under the Datawise / [Datawise.ai](#) brand; “Datumwise” appears to be a product or sub-brand associated with that company.
- **Type:** Privately held company in the IT & services sector.^[10]
- **Focus:** AI, machine learning, data management, data engineering, and building data-intensive applications and platforms.^[11] ^[10]
- **Size and location:** 51–200 employees, headquartered in Atlanta, Georgia, with additional offices in Greece.^[10]

So when you see “[datumwise.ai](#)” in relation to Columna, it’s referring to a product from this data/AI engineering company rather than a separate non-corporate entity.

✱

What evidence exists that Columna's approach works?

There’s very little public, third-party evidence specific to Columna yet (no published customer case studies or benchmarks I can find), but there is strong indirect evidence that its core idea—AI-generated, self-documenting data semantics—is effective, because similar features are now generally available and adopted in mainstream data catalog and metadata tools.^[18]

Direct evidence for Columna

- I’m not finding public **Columna-branded** case studies, metrics, or customer logos that quantify outcomes (e.g., “X% reduction in time to document datasets”) under the Datumwise/Columna name.^[19] ^[20]

- That means any claim that “Columna is proven to work” would mostly have to come from private customer engagements, sales materials, or demos rather than independent documentation.

If you have a specific team or deployment in mind, you’d need internal reports, adoption metrics, or references from Datumwise to see concrete proof.

Indirect evidence: the approach itself

Columna’s core ideas—AI-generated table/column descriptions and semantic labeling propagated through a data stack—are now being adopted in other products:

- **DataHub AI Documentation:** Automatically generates documentation for tables and columns (using column name, type, description, and sample values) and has reached **General Availability** in DataHub Cloud. This indicates both demand and sufficient reliability for production use. ^[18]
- Similar “AI columns” / “prompt columns” features are appearing in analytics databases and business data platforms, where natural-language prompts generate persistent classifications, summaries, and recommendations directly in tables. ^[21]

These tools use the same underlying idea as Columna: LLMs sample data and metadata to create human-readable explanations and labels, reducing manual documentation work. Their GA status and usage in production environments are strong indirect evidence that the pattern is viable. ^{[22] [18]}

What this implies for Columna

From that broader context, you can reasonably infer:

- The **technical approach** (LLM-based descriptions + label suggestions + metadata propagation) is feasible and already in successful use elsewhere. ^{[22] [18]}
- The remaining questions are mostly about **execution quality** (prompt engineering, UI, review workflow, governance controls) and **fit for your stack**, which require either a trial, a pilot, or reference calls with existing customers.

If you tell me what you’d want Columna to improve (e.g., analyst onboarding time, documentation coverage, governance), I can suggest concrete evaluation criteria or experiments you could run to generate your own evidence.

✱

Compare Columna vs dbt Semantic Layer vs Cube.

Columna is an AI-powered *documentation and semantic understanding* layer for your data, while the dbt Semantic Layer and Cube are *queryable metric/semantic layers* that sit between your warehouse and BI/apps. In practice: Columna helps you understand and govern your tables and columns; dbt SL and Cube help tools and applications query consistent metrics over those tables. ^{[34] [35] [36]}

Below is a concise comparison along the main dimensions.

What each one does

- **Columna (Datumwise)**
AI generates human-readable descriptions for tables and columns, suggests governance/classification labels (PII, Finance, Certified, etc.), and propagates that metadata through upstream/downstream views. It's about *self-documenting data* and semantic clarity for humans, not about serving metrics to BI via an API. ^[34]
- **dbt Semantic Layer (MetricFlow)**
Metric layer co-located with your dbt project: you define entities, dimensions, and measures in YAML next to dbt models, and MetricFlow compiles metric queries to SQL when BI tools or APIs ask for them. The warehouse executes the queries; dbt SL standardizes metrics across tools. ^{[35] [37]}
- **Cube**
Standalone semantic layer and data API: engineers author YAML/JS “cubes” (models with measures, dimensions, joins), and Cube exposes them via REST, GraphQL, and SQL APIs, backed by caching and pre-aggregations. It's optimized for embedded analytics and data products. ^{[36] [38] [35]}

Position in the stack

- **Columna**
Sits *on top of* your data warehouse/lake and views. It doesn't replace dbt or Cube; it enriches them by documenting what each dataset means and how it's classified. ^[34]
- **dbt Semantic Layer**
Sits *inside* dbt Cloud, between your dbt models and your BI tools. BI tools call the dbt SL API to get consistent metrics (e.g., revenue, active_users) defined once in dbt. ^{[37] [35]}
- **Cube**
Middleware between data sources and applications. It can import dbt models and build a semantic layer on top, then serve them through APIs with its own caching engine. ^{[38] [39] [36]}

Primary job and consumers

- **Columna**
Job: Make data understandable and governable (docs + labels).
Consumers: Data engineers, analysts, governance teams reading the documentation layer. ^[34]
- **dbt Semantic Layer**
Job: Standardize business metrics and let any connected BI tool query them consistently.
Consumers: BI tools (Hex, Mode, Tableau, etc.) and internal analytics teams needing “define once, query anywhere” metrics. ^{[35] [37]}
- **Cube**
Job: Serve authored data models as high-performance APIs with caching/pre-aggregation.

Consumers: Product/engineering teams building embedded dashboards, data apps, and some internal analytics. ^[36] ^[38] ^[35]

How models are defined

Dimension	Columna	dbt Semantic Layer	Cube	
Core artifact	AI-generated docs + labels	MetricFlow YAML next to dbt models	YAML/JS “cubes” and views	
Authoring style	AI-generated, human-reviewed	Hand-written YAML by analytics engineers	Hand-written YAML/JS by developers	
Main semantics	Column/table meaning + labels	Metrics, entities, dimensions, joins	Measures, dimensions, joins	
Querying	Not a query layer	Metric API, warehouse executes SQL	REST/GraphQL/SQL APIs with caching	^[34] ^[35] ^[36] ^[38]

Strengths and typical use cases

- **Columna – best when:**
 - You struggle with undocumented tables/views and ambiguous column names.
 - You need consistent governance labels and semantic descriptions that propagate through Bronze/Silver/Gold layers. ^[34]
 - You already have a semantic/query layer (dbt SL, Cube, BI) but lack *shared understanding* of the underlying data.
- **dbt Semantic Layer – best when:**
 - You already run dbt Cloud and want metrics defined beside dbt models.
 - Your main consumers are BI tools, and your semantic problem is “multiple tools using different SQL for the same metric.” ^[37] ^[35]
 - You’re okay with the warehouse doing all the heavy lifting, and query volume is dashboard-shaped. ^[35]
- **Cube – best when:**
 - You’re shipping embedded analytics or external data products and need a robust data API with caching/pre-aggregations. ^[40] ^[38] ^[36] ^[35]
 - You have developers to own semantic models, infrastructure, and tuning.
 - You want a standalone semantic layer that can integrate with dbt but doesn’t require dbt.

How they can fit together

They’re not mutually exclusive:

- A common stack could be: **dbt for transformations** → **dbt Semantic Layer or Cube for metrics & APIs** → **Columna for documentation and governance on the same tables/views.**

- Columna improves *human understanding and governance*; dbt SL/Cube improve *queryability and consistency of metrics*.

If you share your current stack (warehouse, dbt yes/no, whether you ship embedded analytics, size of data team), I can give a more opinionated “use X, Y, and maybe Z” recommendation for your situation.

✱

What are Columna's weaknesses?

Columna’s main weaknesses come from the *nature of AI-generated documentation* and from being a relatively new, niche product, rather than from a specific, well-documented failure mode. There isn’t much public, tool-specific criticism or limitation documentation for Columna itself, so what follows is a mix of what we can infer from its design and from known issues with AI-driven data semantics tools. ^[50] ^[51] ^[52]

1. Reliance on AI quality and hallucination risk

Columna samples table data and uses an LLM to generate human-readable descriptions for columns and tables. This introduces several risks: ^[50]

- It can **misinterpret column meaning** or over-generalize from sampled values (e.g., infer “customer ID” when it’s really an internal technical key).
- Like other generative AI systems, it can **hallucinate plausible but wrong descriptions** when schema names are ambiguous or data is sparse. ^[51]
- Documentation quality depends heavily on prompt design, sampling strategy, and human review; without review, you can end up with misleading semantics.

In a governance-sensitive environment, wrong documentation can be worse than no documentation.

2. Need for sustained human curation

Even when AI generates good first drafts, semantic layers and metadata have a known weakness: they require continual maintenance as schemas and metrics change. ^[53] ^[54]

Applied to Columna:

- Every new column, refactor, or upstream model change may require **regenerating and re-validating descriptions and labels**.
- If teams don’t build review workflows and ownership, documentation can quickly become **stale or inconsistent**, just like hand-written docs—only now with an AI layer in the mix.

So Columna reduces the *cost* of documentation, but it doesn’t remove the need for **ownership and review**, which is a common organizational failure point. ^[54]

3. Limited public proof and ecosystem maturity

Compared to dbt Semantic Layer, Cube, or DataHub's AI documentation features, Columna is:

- Less visible in the market: I can't find public customer case studies, benchmarks, or large-scale adoption stories under the Columna brand. ^[55] ^[50]
- Less obviously integrated into a broad ecosystem of BI tools and catalogs.

This implies:

- **Vendor risk:** You'd be betting on a newer niche product, with less public evidence of long-term support and roadmap stability.
- **Integration risk:** More work may be required to knit Columna's documentation into existing catalogs, issue trackers, or governance workflows compared to widely adopted platforms.

4. Not a query or metric layer

By design, Columna is a documentation/semantic understanding layer, not a metric/query layer. ^[50]

Weakness depending on your goals:

- If your main pain point is **inconsistent metrics across tools**, Columna doesn't solve that; you'd still need dbt Semantic Layer, Cube, or similar.
- If you expect it to **enforce semantics in queries** (e.g., preventing misuse of columns in analytic logic), it can only help indirectly via better docs—it doesn't sit in the query path.

That means Columna is additive, not substitutive: some teams may find that it solves “nice to have” problems but leaves core analytics pains unchanged.

5. Potential gaps in lineage and deep governance

Datumwise's broader messaging references column-level lineage and metadata, but it's not clear from public materials how complete or robust Columna's lineage and policy propagation are compared to specialized platforms (e.g., Unity Catalog, DataHub, or enterprise data catalogs). ^[52] ^[56] ^[57]

Potential weaknesses:

- **Incomplete lineage** for complex, non-SQL transformations (Spark code, Python UDFs) is a common issue; tools often only capture lineage inside SQL, which limits how far documentation and labels can be accurately propagated. ^[57]
- **Governance enforcement** (row-level security, masking, policy application) is typically handled by the warehouse/catalog; if Columna only labels and describes but doesn't tie tightly into enforcement, there's a gap between “what the docs say” and “what the system actually does.” ^[52]

6. Organizational adoption challenges

Everything written about semantic layers applies here: the hardest part is **organizational**, not **technical**.^[54]

For Columna, expect:

- You still need **agreement on definitions** and ownership of data domains; AI cannot resolve political disagreements about “what is revenue?” or “what counts as PII.”^[53] ^[54]
- Getting **buy-in from multiple teams** (data engineering, analytics, governance, product) is required for Columna’s docs and labels to become the source of truth, rather than yet another sidecar that some people ignore.^[53] ^[54]

Without that, Columna can degrade into “another tool with auto-generated docs” that don’t materially change how people use data.

If you describe your current pain points (e.g., “analysts can’t understand our Bronze/Silver tables,” “governance is manual and scattered,” “we’re worried about hallucinated docs”), I can turn this into a concrete risk checklist and suggest how to pilot Columna in a way that surfaces these weaknesses early.

*
**

1. <https://dev.to/alexmercedcoder/how-a-self-documenting-semantic-layer-reduces-data-team-toil-21pg>
2. <https://columns.ai/>
3. <https://www.sigmacomputing.com/blog/introducing-ai-columns>
4. <https://www.linkedin.com/company/datawise-ai>
5. <https://docs.caplana.com/data-project-management/common-smart-columns-intro/ai-generated-smart-columns>
6. <https://www.datawiseai.io/copy-of-home-1>
7. <https://datawise.ai/>
8. https://www.youtube.com/watch?v=S_aoc9dExqA
9. https://www.youtube.com/watch?v=qWU3u_2KG-s
10. <https://www.linkedin.com/company/datawise-ai>
11. <https://datawise.ai/>
12. <https://datawisecs.com/>
13. <https://devwise.co/our-company/about-us>
14. <https://datawise-inc.com/>
15. <https://scholarsarchive.library.albany.edu/etd/5/>
16. <https://www.bloomberg.com/profile/company/4529841Z:US>
17. https://link.springer.com/chapter/10.1007/978-3-642-23780-5_34
18. <https://docs.datahub.com/docs/automations/ai-docs>
19. <https://www.sigmacomputing.com/blog/introducing-ai-columns>
20. <https://fivetran.com/docs/activations/datasets/smart-columns/ai-columns>

21. <https://www.youtube.com/watch?v=YwPWYoGsBhY>
22. <https://dev.to/alexmercedcoder/how-a-self-documenting-semantic-layer-reduces-data-team-toil-21pg>
23. <https://www.influxdata.com/blog/what-is-column-database-when-should-you-use-one/>
24. <https://www.danbuglio.com/success-stories>
25. <https://www.youtube.com/watch?v=yg pzKIFt2aM>
26. <https://podcasts.apple.com/us/podcast/scales-of-success-podcast/id1773864140>
27. <https://www.datumate.com/case-studies/>
28. <https://www.fitnessmama.com/podcast/4th-c-section-success-story-with-lauren-brenton-onemamamidwife/>
29. <https://columns.ai/>
30. <https://www.bitwiseglobal.com/insights/3-real-world-customer-case-studies-on-migrating-etl-to-cloud>
31. <https://www.tmswiki.org/forum/tags/success-story/>
32. <https://www.couchbase.com/blog/columnar-database-use-cases/>
33. <https://nationalscoliosiscenter.com/blog/category/success-stories/>
34. <https://dev.to/alexmercedcoder/how-a-self-documenting-semantic-layer-reduces-data-team-toil-21pg>
35. <https://colrows.com/blogs/dbt-semantic-layer-vs-cube-vs-atscale/>
36. <https://unwinddata.com/dbt-semantic-layer-vs-cube>
37. <https://www.getdbt.com/blog/dbt-semantic-layer>
38. <https://david sj.substack.com/p/semantic-layers-a-buyers-guide>
39. https://www.reddit.com/r/dataengineering/comments/170n9rf/cube_integrates_its_semantic_layer_with_dbt/
40. <https://contextflo.com/blog/cube-vs-dbt-vs-contextflo>
41. <https://www.typedef.ai/resources/semantic-layer-architectures-explained-warehouse-native-vs-dbt-vs-cube>
42. https://www.linkedin.com/posts/egorseno_dbt-cubejs-superset-activity-7398705327254409216-cDpx
43. https://www.youtube.com/watch?v=kPNFc-5b_qk
44. <https://www.dremio.com/blog/semantic-layer-tools/>
45. https://www.reddit.com/r/dataengineering/comments/1nfxzrz/what_semantic_layer_products_have_you_used_and/
46. https://www.reddit.com/r/dataengineering/comments/1ayg1r3/semantic_layers_the_good_the_bad_the_ugly/
47. <https://getbruin.com/blog/semantic-layer-tools/>
48. <https://cube.dev/articles/dbt-semantic-layer-alternatives-2026>
49. <https://promethium.ai/guides/top-10-semantic-layer-tools-2026-definitive-comparison/>
50. <https://dev.to/alexmercedcoder/how-a-self-documenting-semantic-layer-reduces-data-team-toil-21pg>
51. <https://www.sei.cmu.edu/blog/weaknesses-and-vulnerabilities-in-modern-ai-integrity-confidentiality-and-governance/>
52. <https://www.starburst.io/blog/what-is-the-semantic-layer/>
53. https://www.youtube.com/watch?v=K_VPF9URZo4&vl=en
54. <https://tryolabs.com/blog/why-semantic-layer-fails-what-works>
55. <https://docs.datahub.com/docs/automations/ai-docs>
56. <https://www.decube.io/post/column-level-lineage-enhancing-data-accuracy-and-governance>

57. https://www.reddit.com/r/databricks/comments/17bco3i/limitations_column_lineage_unity_catalog/
58. <https://www.ibm.com/docs/en/db2/12.1.x?topic=cot-restrictions-limitations-unsupported-database-configurations-column-organized-tables>
59. <https://www.datawiseai.io/>
60. <https://datawise.ai/how-we-deliver>
61. <https://coralogix.com/docs/dataprime/language-reference/limitations/>
62. <https://salesforce.stackexchange.com/questions/309151/what-are-known-limitations-on-number-of-attributes-in-data-extension>
63. https://www.linkedin.com/posts/davidpmariani_where-should-the-semantic-layer-live-why-activity-7374815820104765440-gwj
64. <https://datawise.ai/what-we-do>
65. <https://datawise.ai/>
66. https://www.reddit.com/r/dataengineering/comments/17s7g27/what_are_your_views_on_a_semantic_layer_what_are/